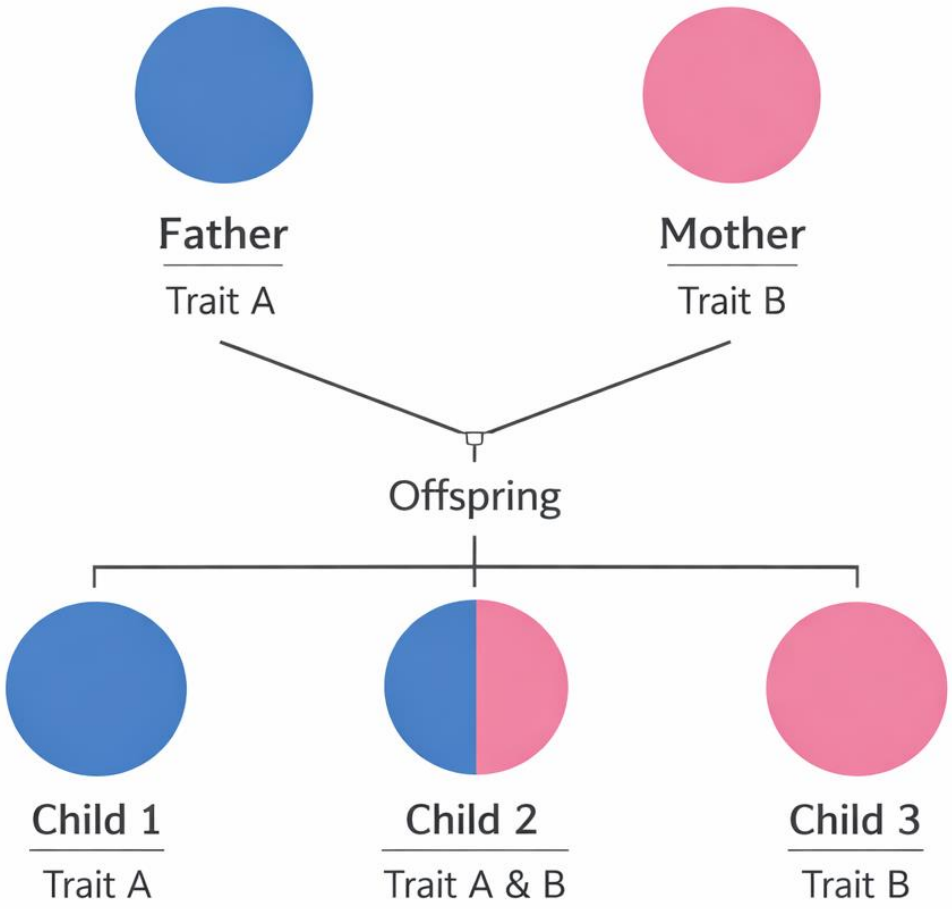
# Module4

## Inheritance and Polymorphism in C++

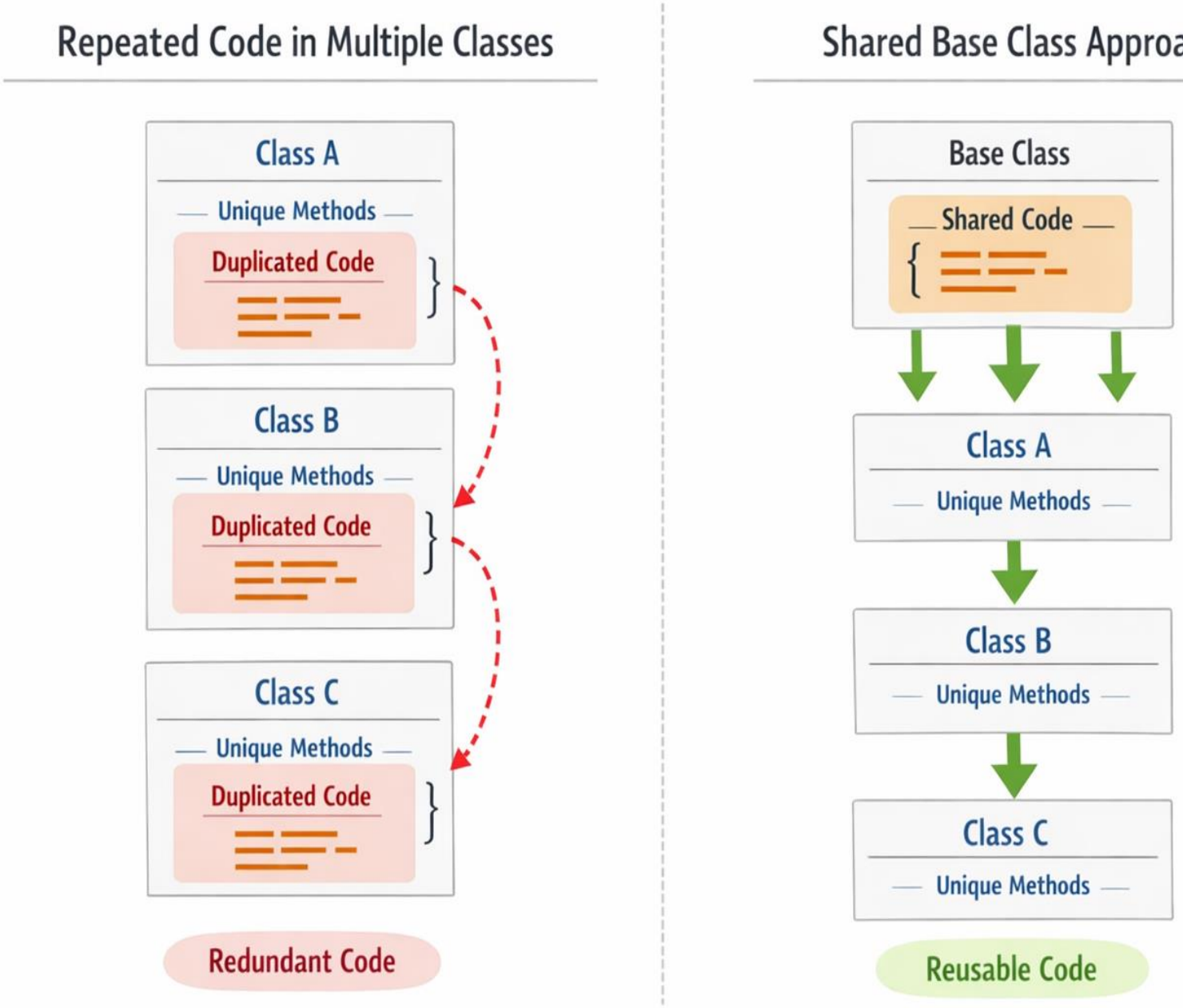
--Building Reusable and Scalable Hierarchies--

# Inheritance




- **Inheritance = Trait Transfer (Parent → Child)**
  Child:
  - gets existing traits
  - can add new traits

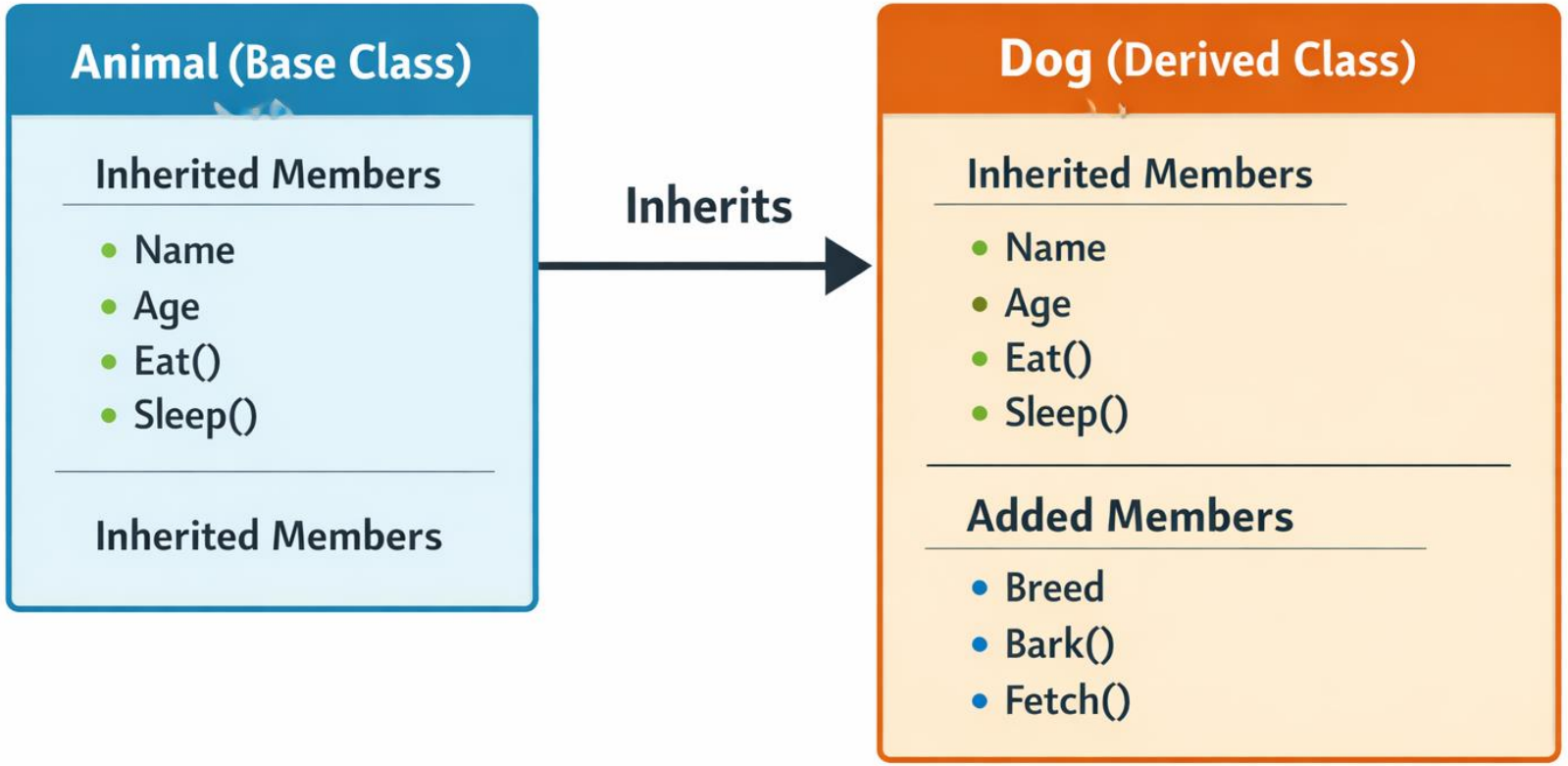
# Why We Need It?




- **Problem: Code Repetition**
  Same common features written again and again.

# Software Mapping

- Parent → Base class
  Child → Derived class
  Traits → Data + Functions



# Inheritance

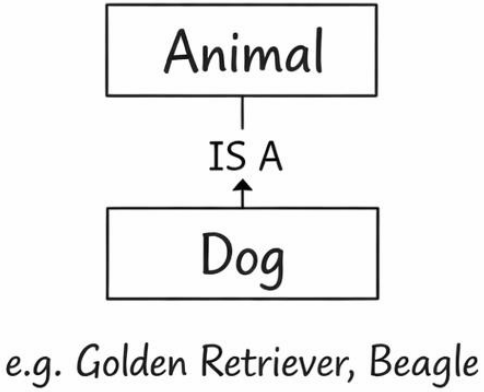
- Inheritance is a mechanism where one class acquires the properties and behavior of another class.
- Base → Derived
  - Example 1: Electronics → Audio Devices → Headphones.
  - Example 2: Account → SavingsAccount / CurrentAccount.
  - Example 3: Employee → FullTimeEmployee / Intern.

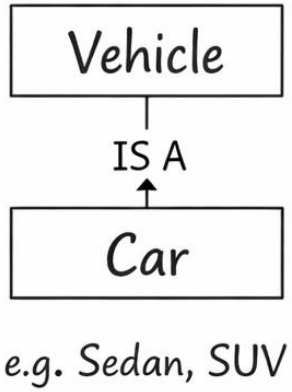
# Why Use Inheritance? (The Business Case)

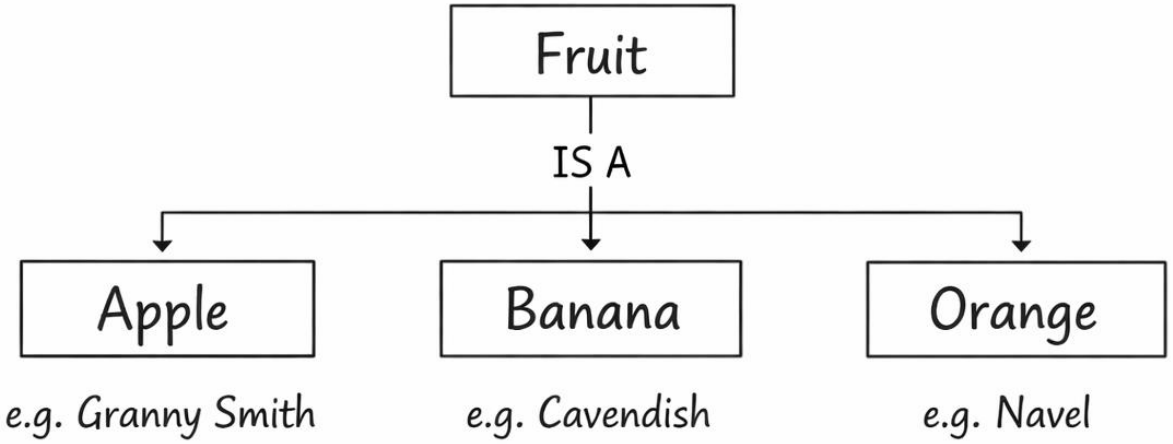
- **Reusability:** Write code once in the Base class, use it in many Derived classes.
- **Extensibility:** Add new features to a Derived class without touching the Base class code.
- **Maintenance:** Fix a bug in the Base class, and it is automatically fixed for all children.
- **Polymorphism Support:** Sets the stage for dynamic binding.

# Relationship

- IS-A Relationship
- Dog IS-A Animal
- Car IS-A Vehicle
- Circle IS-A Shape



# Syntax of Inheritance

```
class Base {
   // Members of Base
};

class Derived : access_specifier Base {
   // Members of Derived
};
```

```
class Device {
public:
    string brand = "Generic";
    void powerOn() { cout << brand << " is starting..."; }
};
class Smartphone : public Device {
public:
    string osType = "Android";
    void showDetails() {
        cout << brand << " runs on " << osType;
    }
};
```

# Memory Layout of a Derived Object

- When an object of Smartphone is created, memory is allocated for BOTH Device members and Smartphone members.

- The Base class part is constructed first.

# What Are Access Specifiers?

- **Access Specifiers control visibility of class members.**
- They define, who can access variables and functions?
- Three types: <u>public, private and  protected</u>
  - public: Accessible everywhere (inside the class, by derived classes, and by the outside world/main).
  - private: Accessible only inside the same class. Derived classes cannot access these directly.
  - protected: Accessible inside the class AND by derived (child) classes. It is only hidden from the "outside" world (main).

- public (The Open Door): Anyone can see it. You can access it from inside the class, from a child class, and from a random function like main().

- private (The Personal Safe): ONLY the class that owns it can see it. Not even its own children can touch it.

- protected (The Family Secret): This is the "middle ground." It’s hidden from the outside world (like private), but it is wide open to any child class (like public).

```
#include <iostream>
#include <string>
using namespace std;
// Base Class
class Human {
public:
   string name;
   void breathe() {
      cout << name << " is breathing..." << endl;
   }
};
```

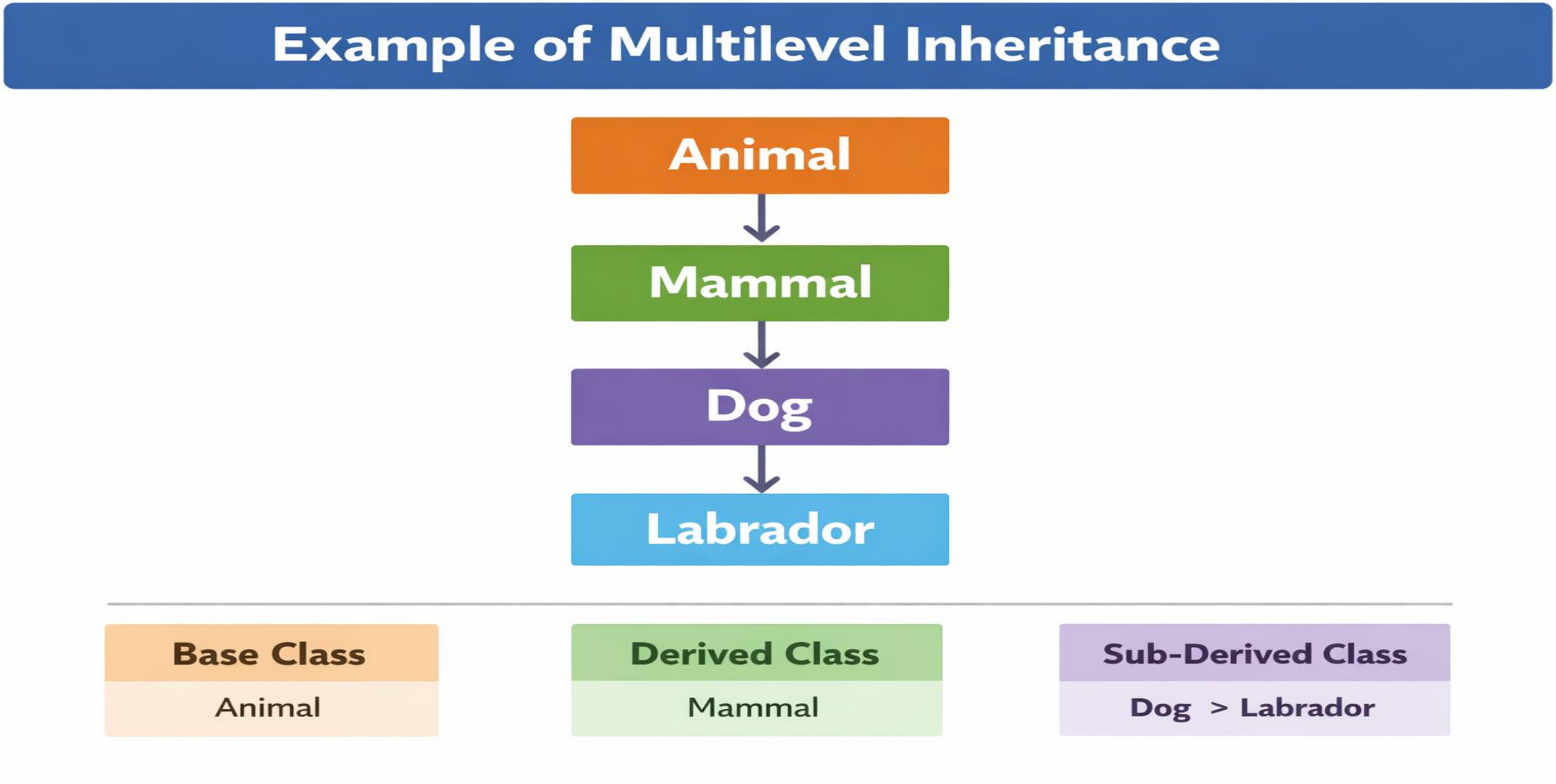
```
// Derived Class (Single Inheritance)
class Programmer : public Human {
public:
   string language;
   void code() {
      cout << name << " is coding in " << language << "!" << endl;
   } };
int main() {
   Programmer p1;
   p1.name = "Alice";     // Inherited from Human
   p1.language = "C++";   // Defined in Programmer
   p1.breathe();          // Calling Base function
   p1.code();             // Calling Derived function
   return 0;
}
```

```
#include <iostream>
using namespace std;
class Shape {
protected:
   float width, height; // Family secrets:
//only Shape and children see this
public:
   void setValues(float w, float h) {
      width = w;
      height = h;
   } };
class Rectangle : public Shape {
public:
   float area() {
      return width * height; // Accessing
//protected members of Parent
   } };
```

```
int main() {
   Rectangle rect;
   rect.setValues(10.5, 5.0); // Setting
//data through Base method
   cout << "Area of Rectangle: " <<
rect.area() << endl;
   // rect.width = 10; // ERROR!
//protected members are hidden from
//main()
   return 0;
}
```

// Single Inheritance

- Multilevel Inheritance - A class is derived from a class which is also derived from another class.



```
class BasicPhone { public: void call() { cout << "Calling..."; } };
class FeaturePhone : public BasicPhone
{public: void takePhoto() { cout << "Photo taken."; } };
class SmartPhone : public FeaturePhone
{public: void browse() { cout << "Browsing Web..."; }};
int main() {    // Creating an object of the most derived class
  SmartPhone myPhone;
  myPhone.call(); // 1. Accessing a function from the Grandparent (BasicPhone)
  myPhone.takePhoto(); // 2. Accessing a function from the Parent (FeaturePhone)
  myPhone.browse(); // 3. Accessing a function from its own class (SmartPhone)
   return 0;
}
// Multilevel Inheritance
```

# Access Specifier Vs Access Filters

```
class BasicPhone { public: void call() { cout << "Calling..."; } };
class FeaturePhone : public BasicPhone
```

| Scenario | Access Specifier – Member Born | Access Filter – Member Refer (Inherit) |
|---|---|---|
| 1 | Public | : public |
| 2 | Public | : protected |
| 3 | Public | : private |
| 4 | Protected | : public |
| 5 | Protected | : protected |
| 6 | Protected | : private |
| 7 | Private | : public |
| 8 | Private | : protected |
| 9 | Private | : private |

# Access Filters

- **1. The "Ceiling" Rule (Answering your Public question)**
- If a member is declared as **Public** in the Parent, it is "Wide Open." But when the Child inherits it, the Child can "tighten" the security.
- **Public Inheritance:** The Child says, "I'll keep it as it is." (Result: **Public**)
- **Protected Inheritance:** The Child says, "I want my kids to see it, but not the neighbors (main)." (Result: **Protected**)
- **Private Inheritance:** The Child says, "I'm keeping this for myself only. My kids won't even see it." (Result: **Private**)
- **The Limit:** You can **never** go higher than the Inheritance Mode you chose. If you inherit protected, even a public variable from the parent becomes protected in the child.

# The scope breakdown

| Scenario | Access Specifier – Member Born | Access Filter – Member Refer | Result in Parent (Child) | Result in Child (Grandchild) |
|---|---|---|---|---|
| 1 | Public | : public | **Visible** | **Visible** |
| 2 | Public | : protected | **Visible** | **Visible** |
| 3 | Public | : private | **Visible** | **Restricted** |
| 4 | Protected | : public | **Visible** | **Visible** |
| 5 | Protected | : protected | **Visible** | **Visible** |
| 6 | Protected | : private | **Visible** | **Restricted** |
| 7 | Private | : public | **Restricted** | **Restricted** |
| 8 | Private | : protected | **Restricted** | **Restricted** |
| 9 | Private | : private | **Restricted** | **Restricted** |

```
#include <iostream>
#include <string>
using namespace std;
// Level 1: Grandparent
class Person {
protected:
   string name = "John Doe"; // Visible to all descendants
};
// Level 2: Parent (Inherits Person)
class Employee : public Person {
protected:
   int employeeID = 101;
};
// Level 3: Child (Inherits Employee)
class Manager : public Employee {
public:
   void showDetails() {
      // Can see both Grandparent and Parent protected data
cout << "Manager Name: " << name << endl;
cout << "Employee ID: " << employeeID << endl;
   } };
int main() {
   Manager m;
   m.showDetails();
   return 0;
}
// Multilevel Inheritance
```

```
#include <iostream>
using namespace std;
// Level 1: Basic Console
class Console {
public:
void powerOn()
{ cout << "System Starting..." << endl; }
};
// Level 2: Handheld (Adds a Screen)
class Handheld : public Console {
public:
void enableScreen() { cout << "Displaying
on built-in screen." << endl; }
};
```

```
// Level 3: ProHandheld (Adds 4K Graphics)
class ProHandheld : public Handheld {
public:
void enable4K()
{ cout << "Ultra HD Graphics enabled!" <<
endl; }
};
int main() {
ProHandheld myDevice;
myDevice.powerOn();  // From Grandparent
// From Parent
myDevice.enableScreen();
myDevice.enable4K();    // From Child
return 0;
}

// Multilevel Inheritance
```

# Hierarchical Inheritance

- One parent, many children.
- This is the opposite of Multiple Inheritance. Here, **one Base Class** serves as the parent for **multiple Derived Classes**. This is used to create specialized versions of a general concept.
- **Analogy**: A Bank (Base) has different types of accounts like Savings and Current (Children).

```
// Hierarchical Inheritance
#include <iostream>
using namespace std;
class OperatingSystem {
public:
void boot()
{ cout << "System is booting" << endl; }
};
// Child 1
class Android : public OperatingSystem {
public:
void playStore()
{ cout << "Accessing Google Play Store."
<< endl; }
};
// Child 2
class iOS : public OperatingSystem {
public:
void appStore()
{ cout << "Accessing Apple App Store."
<< endl; }
};
int main() {
Android phone1;
iOS phone2;
phone1.boot();
// Both share the same base function
phone2.boot();
return 0;
}
```

# Multiple Inheritance

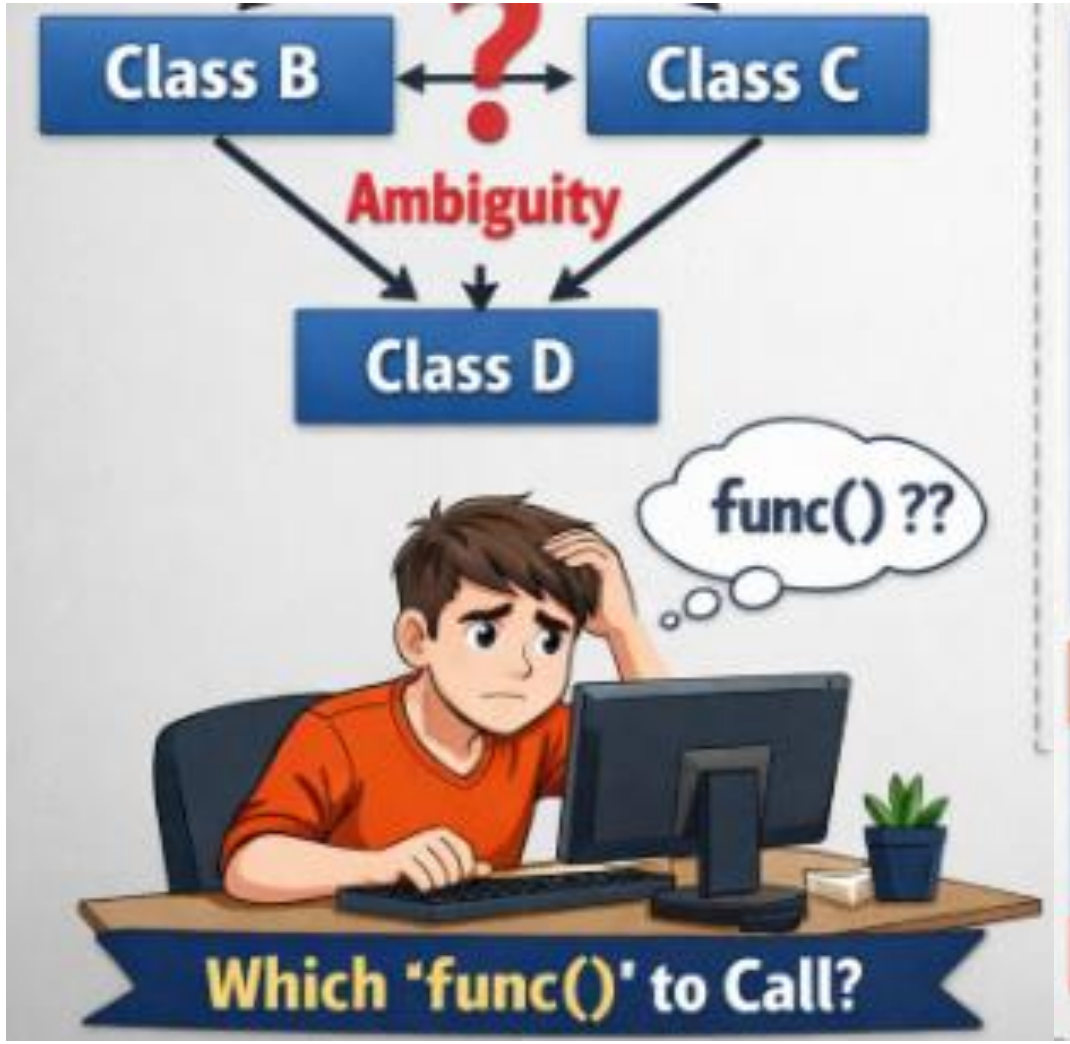
- A single child class derived from multiple direct parents.
- In this scenario, one **Derived Class** inherits from **two or more Base Classes**. Think of a SmartPhone inheriting features from both a Phone class and a Camera class simultaneously.

- **Syntax**: class Derived : public Base1, public Base2 { … };.

```
// Multiple Inheritance: 1 child, 2 parents

#include <iostream>
using namespace std;
class Phone {
public:
void makeCall()
{ cout << "Making a voice call..." << endl; }
};
class Camera {
public:
void takePhoto() { cout << "Capturing a high-res photo..." << endl; }
};
class SmartPhone : public Phone, public Camera {
public:
 void useApps()
{ cout << "Opening mobile apps..." << endl; }
};
int main() {
SmartPhone myDevice;
myDevice.makeCall();  // From Phone
myDevice.takePhoto(); // From Camera
myDevice.useApps();   // From SmartPhone
return 0;
}
```

# Ambiguity




**Both B and C have func():**

```
class B {
    void func() {
        cout << "Hello from B"}
}
class C {
    void func() {
        cout << "Hello from C"}
}
```

**Ambiguous Call:**

```
D obj;
obj.func();
```

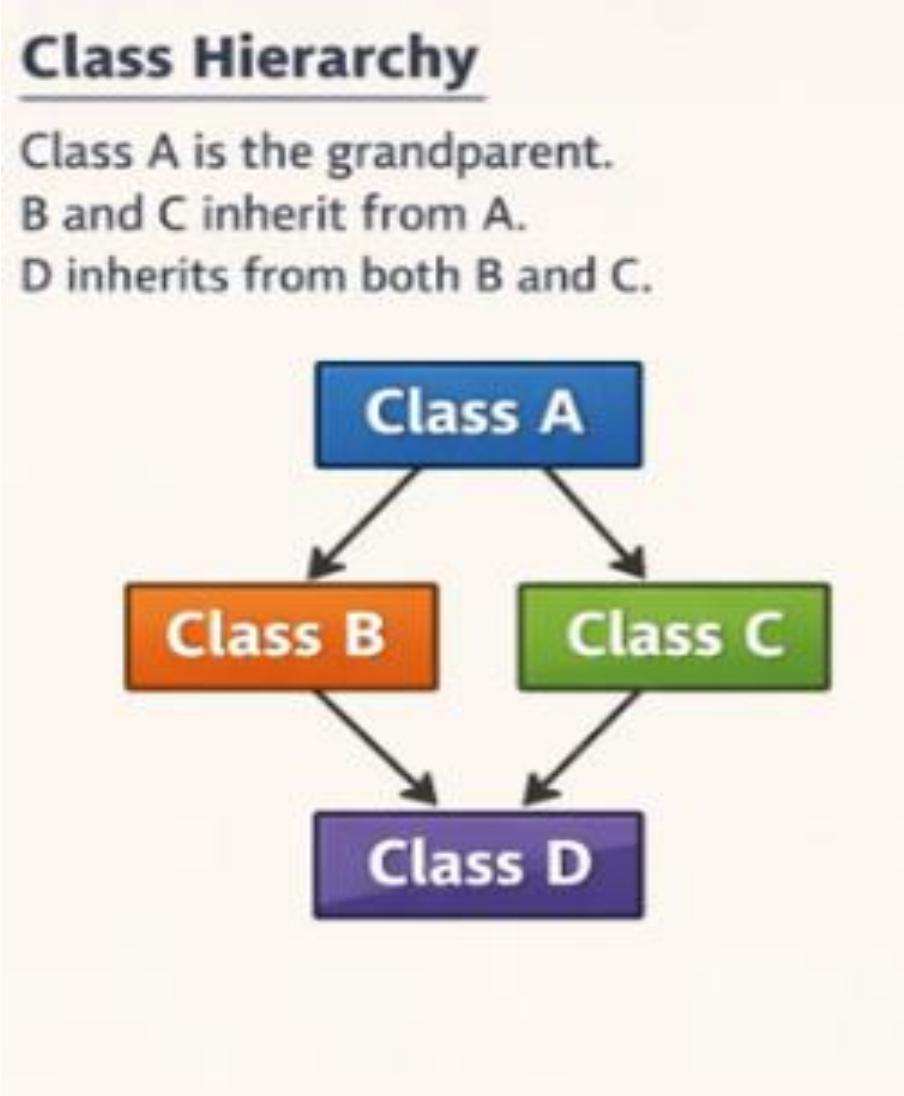
**Error: 'func' is ambiguous!**

# Multiple  Inheritance - Ambiguity

- If ParentA and ParentB both have a function called start(), and the Child tries to call obj.start(), the compiler will throw an error: **"request for member 'start' is ambiguous."**
- **The Scope Resolution Operator (::)**
  - This is the most common fix. You explicitly tell the compiler which parent’s version to use.

```
int main() {
    Child obj;
    // obj.start();          // ERROR: Ambiguous!
    obj.ParentA::start();     // FIXED: Calls Parent A's version
    obj.ParentB::start();     // FIXED: Calls Parent B's version
}
```

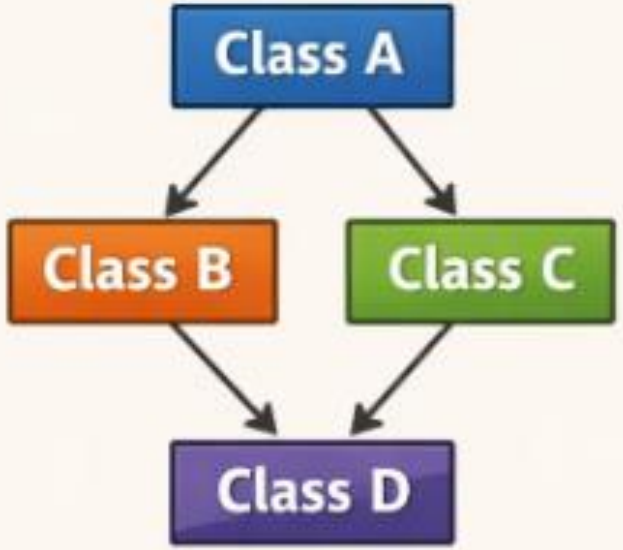
# Multipath Inheritance



# The Diamond Problem

## Class Hierarchy

Class A is the grandparent.
B and C inherit from A.
D inherits from both B and C.

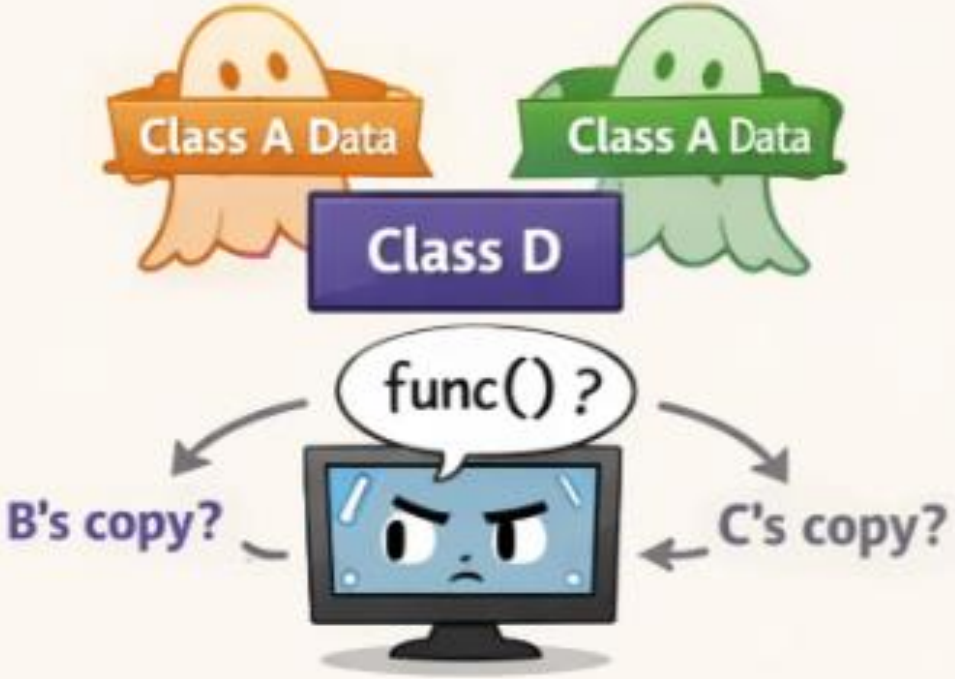



## The Issue:

Class D gets two copies of Class A's data.

## The Ambiguity:

Which version of **func()?**
Call from B or call from C?



# Multipath Inheritance & Ambiguity

- This is the most advanced "Type" of inheritance.
- It occurs when a class is derived from other classes which share a common base. This often leads to the **"Diamond Problem"**.
- **The Issue**: If Class A is a grandparent, and Classes B and C inherit from A, and then Class D inherits from both B and C, Class D gets **two copies** of Class A's data.
- **The Ambiguity**: When Class D tries to call a function from Class A, the compiler gets confused: "Should I use the copy from B or the copy from C?".

# Multipath Ambiguity Solution: The virtual Keyword

- To fix this, you must use the virtual keyword when the **Parents** inherit from the **Grandparent**. This tells the compiler to share a single instance of the Grandparent class among all derived classes.

```
class A {
public:
    void display() { cout << "Hello from A"; }
};
// Use 'virtual' to ensure only one copy of A exists in the chain
class B : virtual public A { };
class C : virtual public A { };
class D : public B, public C { };
int main() {
    D obj;
    obj.display(); // FIXED: No longer ambiguous!
    return 0;
}
```

| Inheritance Type | Conflict | Handling Method |
| --- | --- | --- |
| **Multiple** | Two different parents have the same function name. | Use **Scope Resolution** ( `obj.ParentB::func()` ). |
| **Multipath** | Two paths lead to the same Grandparent. | Use **Virtual Inheritance** ( `class B : virtual public A` ). |

# Inheritance Conflicts

# Constructors in Inheritance - Rule

- When you create an object of a derived class, C++ does not start with that class. It builds the foundation first.
- **Order of Construction**: The **Base class** constructor is executed first, followed by the **Derived class** constructor.
- **The Logic**: A child cannot exist without its parents. The base class must be initialized first so the derived class has something to build upon.

# Constructor Order - Multilevel Inheritance (The Chain)

- For BasicPhone → FeaturePhone → SmartPhone
- BasicPhone() (Grandparent)
- FeaturePhone() (Parent)
- SmartPhone() (Child)

# Constructor Order - Multiple Inheritance (The Merge)

- If a class has two parents, like class
  - SmartPhone : public Phone, public Camera:
- **Order of Declaration**: Constructors are called in the order they appear in the class header.
- **Header**: class SmartPhone : public Phone, public Camera
- **Result**: Phone() runs first, then Camera() runs, and finally SmartPhone() runs.

# Passing Arguments to Base Constructors

- If the Parent class needs a value (like a phone number or ID), the Child class must "pass" that value up using an **Initialization List**.

```
class Parent {
public:
   Parent(int id) { cout << "Parent ID: " << id; }
};

class Child : public Parent {
public:
   // The Child constructor "forwards" the ID to the Parent
   Child(int id, int age) : Parent(id) {
      cout << "Child Age: " << age;
   }
};
```

# Virtual Inheritance (The Diamond Fix)

- In a **Multipath (Diamond)** scenario, the virtual keyword changes the rules again.
- The **Virtual Base Class** (the Grandparent) is initialized **only once**.
- It is initialized by the **most derived class** (the Grandchild) directly, skipping the intermediate parents to ensure no duplicate data is created.

```
#include <iostream>
using namespace std;
class Phone {// Parent 1
public:
   Phone() { cout << "1. Phone part initialized." << endl; } };
class Camera {// Parent 2
public:
   Camera() { cout << "2. Camera part initialized." << endl; } };
// Child inheriting from both & Changing the order here changes the execution order!
class SmartPhone : public Phone, public Camera {
public:
   SmartPhone() { cout << "3. SmartPhone (Child) initialized." << endl; } };
int main() {
   cout << "--- Creating a SmartPhone Object ---" << endl;
   SmartPhone myS24;
   return 0;
}
```

# Polymorphism

- One interface, multiple behaviour
- A **payment system** can process:
  - Credit card payment
  - UPI payment
  - Net banking payment
- All use the same function:
- processPayment()
- But behavior differs.

# Types of Polymorphism in C++

| Type | When resolved |
|---|---|
| **Compile time polymorphism** | At compilation |
| **Dynamic polymorphism** | At runtime |

- Compile time
  - Function overloading
  - Operator overloading
- Dynamic
  - Function overriding
  - Virtual functions

# Function Overloading: The "Multi-Tasker"

- Inside the same Payment class, you have one "Pay" button, but it needs to behave differently based on what information the user provides.
  - Scenario A: User provides (int creditCardNumber) → The system processes a Card Transaction.
  - Scenario B: User provides (string upiID) → The system opens a UPI app.
  - Scenario C: User provides (int amount, string couponCode) → The system deducts the discount first.

Why it’s needed: It provides convenience. Instead of having 50 different function names like payWithCard(), payWithUPI(), and payWithCoupon(), the programmer uses one simple name: pay(). The compiler automatically picks the right one based on the input.

```
#include <iostream>
#include <string>
using namespace std;
class Notification {
public:
// Overload 1: Send Text
void send(string message) {
cout << "Sending Text: " << message << endl;   }
// Overload 2: Send Photo with Caption
void send(string fileName, string caption) {
cout << "Sending Photo: " << fileName << " with caption: " << caption << endl;
 }
// Overload 3: Send Location (GPS Coordinates)
 void send(double lat, double lon) {
cout << "Sending Location: " << lat << ", " << lon << endl;
 } };
int main() {
Notification messenger;
// Calls Overload 1
messenger.send("Hello!");
// Calls Overload 2
 messenger.send("vacation.jpg", "Fun!");
// Calls Overload 3
messenger.send(12.91, 79.13);        // Calls Overload 3
   return 0;
}
```

//FUNCTION OVERLOADING

# Operator Overloading

- **Operator Overloading** is a type of polymorphism where we give special meaning to standard operators (like +, -, *, ==, etc.) so they can work with user-defined objects.
- Without it, the compiler only knows how to add basic data types like int or float. Operator overloading allows us to extend that logic to complex, real-world objecets

# OO - How?

- Imagine the + symbol is a **Universal Connector**. Its behavior changes based on what you are trying to connect:
- **With Numbers:** If you give it 5 + 5, it acts like a **Calculator** and gives you 10.
- **With Strings (Text):** If you give it "Hello" + "World", it acts like **Glue** and gives you "HelloWorld".
- **With Water Pipes:** If you "overload" it for a Pipe class, Pipe1 + Pipe2 would act like a **Coupler**, creating one long continuous pipe.
- **With a Shopping Cart:** If you have Cart A (with an Apple) and Cart B (with an Orange), Cart A + Cart B acts like a **Merger**, resulting in a single cart containing both fruits.
- **The Theory:** The symbol remains the same (+), but the **logic** inside changes depending on the "operands" (the objects) it is touching.

# Rules of Operator Overloading

- **No New Symbols:** You cannot create a new operator like @ or $. You can only redefine existing ones.
- **Precedence Stays Same:** Even if you overload *, it will still be calculated before + in an equation.

- return_type operator symbol (arguments) { // Logic for the operation }

```
#include <iostream>
using namespace std;
class Time {
public:
  int hours, minutes;
  Time(int h, int m) { hours = h; minutes = m; }
// Overloading the '+' operator to add two Time objects
   Time operator + (const Time&t)
 {
 int h = hours + t.hours;
 int m = minutes + t.minutes;
 if (m >= 60) {h++; m -= 60;}
  return Time(h, m);
 }
void display()
{ cout << hours << "h " << minutes << "m" << endl; }
};
```

```
int main() {
 // 2 hours 45 mins
Time shift1(2, 45);
// 1 hour 30 mins
Time shift2(1, 30);
// Using the overloaded '+'
Time totalWork = shift1 +
shift2;
// Result: 4h 15m
totalWork.display();
return 0;
}
```

```
#include <iostream>
using namespace std;
class Distance {
public:
   int feet;    int inches;
   // Constructor using the "Modern" Initializer List
   Distance(int f, int i) : feet(f), inches(i) {}
   // Operator Overloading
   Distance operator + (const Distance& d)
{     int f = feet + d.feet;      // Add the feet
      int i = inches + d.inches;  // Add the inches
// If inches are 12 or more, convert to feet
      if (i >= 12) {
        f += (i / 12); // Full feet are in the inches
         i = i % 12; // Keep the remaining inches
    }  return Distance(f, i); }  // Return result obj
   void display() {
      cout << feet << " feet, " << inches << " inches" << endl;
   }
};
int main() {
Distance plank1(3, 9);
Distance plank2(2, 8);
// Using the overloaded '+' operator
Distance totalLength = plank1 + plank2;
cout << "Total Length: ";
totalLength.display(); // Result: 6 feet, 5 inches
return 0;
}
```

```
#include <iostream>
#include <cstring> // For traditional C-style string functions
using namespace std;
class MyString {
public:
   char str[100]; // Traditional character array
   MyString() {strcpy(str, ""); }
      MyString(char s[]) {strcpy(str, s); }
   // Overloading '+' to join two strings
   MyString operator + (const MyString& s)
{ MyString temp;
strcpy(temp.str, str);      // Copy the first string
strcat(temp.str, " ");      // Add a space inbet.
strcat(temp.str, s.str);    // Append the 2nd string
 return temp;    }
```

```
void display() {
      cout << "Result: " << str << endl;
   } };
int main() {
MyString name1("Steve");
MyString name2("Jobs");
// Using '+' to join objects instead of calling name1.join(name2)
MyString fullName = name1 + name2;
fullName.display(); // Output: Steve Jobs
   return 0;
}
```

# Operators for OO

| Category | Operators | Possible Real-Time Operations |
|---|---|---|
| **Arithmetic** | +, -, *, /, % | Adding Time, subtracting Dates, or multiplying Matrix data. |
| **Relational** | ==, !=, <, >, <=, >= | Comparing if two User objects are the same or sorting Product prices. |
| **Assignment** | =, +=, -=, *=, /= | Updating a BankBalance or appending a String. |
| **Unary** | ++, --, - (negation) | Incrementing a Counter or toggling a "Mute" status on a Media object. |
| **Logical** | &&, \|\|, ! | Define how your objects should be evaluated in "True/False" conditions or combined logic. |
| **Subscript** | [] | Accessing specific elements in a custom Database or Array class. |
| **Input/Output** | <<, >> | Custom printing of a Student profile or reading Sensor data from a stream. |
| **Function Call** | () | Making an object behave like a function (called "Functors"). |

# Operators Cannot be Overloaded

| Operator | Reason |
|---|---|
| :: | scope resolution |
| . | member access |
| ?: | conditional operator |
| sizeof | compile-time operator |

# Function Overriding: The "Software Update"

- The Operating System (OS) Update
  - This happens across Inheritance. Imagine a Base Class called OS_Version_1 and a Derived Class called OS_Version_2.

- Base Class (v1): Has a function unlockPhone(). It uses a simple "Swipe to Unlock" animation.

- Derived Class (v2): Inherits everything from v1, but it wants to "Upgrade" the security. It Overrides the unlockPhone() function to use "Face ID" instead.

- Why it’s needed: It allows for Evolution. You don't want to change the name of the function to unlockWithFaceID() because all the other apps on the phone are already programmed to call unlockPhone(). By overriding, the "Trigger" stays the same, but the "Action" becomes modern.

# Overriding - Where is it "Seriously" Needed?

- Imagine a **Central Bank (Base Class)** that sets a rule: calculateInterest().
- **The Problem:** The Central Bank defines a default interest rate of **4%**.
- **The Reality:** Every specific bank (SBI, HDFC, ICICI) has its own rate.
- **The Solution (Overriding):**
  - class SBI : public CentralBank overrides it to **6%**.
  - class ICICI : public CentralBank overrides it to **7%**.
- **Without Overriding:** If you had a list of 1,000 different bank objects, you would have to write a massive if-else statement to check which bank it is before calling its specific interest function.
- **With Overriding:** you just call bank->calculateInterest() and C++ automatically runs the correct version for that specific bank. This is called **Runtime Polymorphism**.

```
#include <iostream>
using namespace std;
class CentralBank {
public:
void getInterestRate() {
cout << "General Interest Rate: 4%" << endl;
} };

class SBI : public CentralBank {
public:
// OVERRIDING the parent's function
void getInterestRate() {
cout << "SBI Special Interest Rate: 6.5%" << endl;
} };
```

```
int main() {
CentralBank genericBank;
SBI sbiBank;
// Parent version
genericBank.getInterestRate();
 // Child version (Upgraded)
sbiBank.getInterestRate();
return 0;
}
```

# Overloading Vs Overriding

| Concept | What is it? | Real-time Example |
| --- | --- | --- |
| **Function Overloading** | **Same name, different inputs.** | Typing a name vs. a number in a search bar. |
| **Operator Overloading** | **Same operator, new behavior.** | Using `+` to join two strings or two "Cart" objects. |
| **Function Overriding** | **Same name, same inputs, different logic.** | A "Pro" version of an app changing how the "Save" button works. |

# What is Function Overriding?

- Function overriding happens when
- Base class has a function
- Derived class provides **its own implementation**
- Conditions:
  - ✓ Same function name
    ✓ Same parameters
    ✓ Inheritance required

```
class Animal
{
public:
   void sound()
   {       cout<<"Animal makes sound";  } };

class Dog : public Animal
{
public:
   void sound()
   {      cout<<"Dog barks"; } };

Dog d;
d.sound();
```

- C++ uses **static binding**.
- Meaning:
- Decision happens **at compile time**.

```
#include <iostream>
using namespace std;
class Animal
{
public:
   void speak()   {  cout << "Animal sound" << endl; } };
class Dog : public Animal
{
public:
   void speak()  { cout << "Dog barks" << endl; } };
int main()
{
   Animal *a;
   Dog d;    a = &d;
   a->speak();
   return 0;  }
```

- Animal :: speak();
- Output: Animal sound

# Why?

Even though `a` points to a `Dog` object, the function call is decided using the **pointer type**:

```
Animal *a;
```

So compiler binds:

```
Animal::speak()
```

This is called:

- **Early binding**
- **Static binding**
- **Compile-time binding**

- a points to Dog object ✓
- But function selection is still based on base pointer type ❗

```
#include <iostream>
using namespace std;
class Animal
{
public:
   virtual void speak()
  {     cout << "Animal sound" << endl; } };
class Dog : public Animal
{
public:
   void speak()
   { cout << "Dog barks" << endl; } };
class Cat : public Animal
{
public:
   void speak()
   { cout << "Cat meows" << endl; } };
```

```
int main()
{
    Animal *a;
    Dog d;
    Cat c;
    a = &d;
    a->speak();
    a = &c;
    a->speak();
    return 0;
}
```

<u>Output</u>

Dog barks
Cat meows

# Dynamic Polymorphism

- **The "Save As..." System (The OS Problem)**
- **The Challenge:** You are designing a Photo Editor. You have a "Save" button.
- A **.JPG** file needs "Lossy Compression."
- A **.PNG** file needs "Transparency Mapping."
- A **.PDF** file needs "Vector Embedding."
- **Efficient way to handle this?**

# Dynamic Polymorphism

- **Dynamic Polymorphism** (also known as Late Binding or Runtime Polymorphism) allows a single interface (a pointer) to handle different types of objects, and the program decides **which** function to call only when it is actually running.
- **Static** = Decisions made while writing/compiling code.
- **Dynamic** = Decisions made while the "Play" button is pressed.
- A virtual function is a member function that you expect to be redefined in derived classes.
- The Message: It tells the compiler: "Wait until Runtime to decide which function to call."
- Syntax: virtual void functionName();

- To achieve this, you need three specific ingredients:
- **Inheritance**: A Base and a Derived class.
- **Virtual Function**: The keyword virtual in the Parent class.
- **Pointer**: A pointer of the **Base class** type.

```
class Parent {
public:
   virtual void show() {
      // Parent logic
   } };
class Child : public Parent {
public:
   void show() {
     // override  the Child logic
   } };
```

```
#include <iostream>
using namespace std;
// Base Class
class Shape {
public:
// Virtual function: Tells C++ to look at the object    virtual void draw() {
cout << "Drawing a shape..." << endl;
 } };
// Derived Class 1
class Circle : public Shape {
public:
void draw() override {
cout << "Drawing a Circle (O)" << endl;
 }};
```

```
// Derived Class 2
class Rectangle : public Shape {
public:
 void draw() override {
cout << "Drawing a Rectangle " << endl;
} };
int main() {
Shape* s;  // Base class pointer
Circle c;    // Actual Object 1
Rectangle r;  // Actual Object 2
s = &c; // Scenario 1: Point to Circle
s->draw(); // Drawing a Circle
s = &r; // Scenario 2: Point to Rectangle
s->draw(); // Drawing a Rectangle
return 0; }
```

# Key points

- **Without virtual (Static Binding):** The compiler says, This is a Shape pointer, so I will call the Shape version of the function.
- **With virtual (Dynamic Binding):** The compiler says, "This is a Shape pointer, but it is marked virtual. I must look at the Actual Object (the Circle or Rectangle) to find the correct version of the function.“
- The program becomes "smart." It looks past the pointer and sees the actual object (Circle or Rectangle) sitting in memory.
- **The Benefit**: You can create an array of Shape* and store 100 different shapes. One simple loop shapes[i]->draw() will draw everything correctly without a single if-else statement.

# Destructors

- A destructor is a special function used to destroy objects and release resources.

Syntax:

```
~ClassName()
```

Example:

```
class Animal
{
public:
   ~Animal()
   {       cout << "Animal Destructor" << endl;  } };
```

- Destructor runs when object goes out of scope or is deleted.

- When we use **dynamic polymorphism**, we usually create objects like this:
  - Animal *a = new Dog();
- Later we delete it:
  - delete a;
- Now the question is: Which destructor should run?
  - Dog destructor
  - Animal destructor
  - Both?

```
#include <iostream>
using namespace std;
class Animal
{
public:
   ~Animal()
   {       cout << "Animal destructor called" << endl;} };
class Dog : public Animal
{
public:
   ~Dog()
   {       cout << "Dog destructor called" << endl; } };
int main()
{
   Animal *a = new Dog();
   delete a;
}
```

- Animal destructor called
- So resources inside Dog may **not be released**.

```
#include <iostream>
using namespace std;
class Animal
{
public:
   virtual ~Animal()
   { cout << "Animal destructor called" << endl; } };
class Dog : public Animal
{
public:
   ~Dog()
   { cout << "Dog destructor called" << endl; } };
int main()
{
   Animal *a = new Dog();
   delete a;
}
```

- Dog destructor called
- Animal destructor called

```
#include<iostream>
using namespace std;
class Shape
{
public:
   virtual void draw() = 0;
};
class Circle : public Shape
{
public:
   void draw()
   {       cout << "Drawing Circle" << endl; } };
int main()
{   Circle c;     c.draw(); }
```

# Abstraction

- Showing only **essential features** while hiding **implementation details**.
- Example: - When you drive a **car** you use:
  - Steering
  - Brake
  - Accelerator
- You **will not worry** how the engine works internally.
- You only know **what to do**, not **how it works**.
- That is **abstraction**.

- In OOP, abstraction is implemented using:
  - **Pure virtual functions**
- A base class defines **what functions must exist**, but **not how they work**.
- Derived classes provide the **actual implementation**.

- Rules involved in house  construction
- Car Manufacturing
- TV Manufacturing

# Pure Virtual Function

- A **pure virtual function** is a function declared in a base class **without implementation**.

- virtual return_type function_name() = 0;

- virtual void draw() = 0;

- The =0 means:
  - No implementation in base class
  - Derived classes must implement it

```
#include<iostream>
using namespace std;
class Shape
{
public:
   virtual void draw() = 0;
};
class Circle : public Shape
{
public:
   void draw()
   {  cout << "Drawing Circle" << endl; } };
int main()
{
   Circle c;
   c.draw();
}
```

```
#include <iostream>
using namespace std;
class Course
{
public: // pure virtual function
   virtual void conductClass() = 0;
};
class Programming : public Course
{
public:
   void conductClass()
   { cout << “Lab session" << endl; } };
class Mathematics : public Course
{
public:
   void conductClass()
 { cout << “Problem solving class" << endl; } };
```

```
int main()
{
   Course *c;      // base class pointer
   Programming p;
   Mathematics m;
   c = &p;
   c->conductClass();
   c = &m;
   c->conductClass();
   return 0;
}
```